



Universidad Técnica Estatal de Quevedo
Aplicaciones Distribuidas – Séptimo A

TiendaTech

Documento acumulativo E1–E4
Arquitectura, implementación y evaluación experimental

Proyecto Fin de Curso – Paso 13

Integrante	Rol asignado
Jhinson Stalyn Aucatoma Celorio	Arquitecto
Jeremy Ruperto Gaibor Rodriguez	Desarrollador
Andy Paul Sanchez Pilaloe	Responsable de Calidad
José Alejandro Lozano Morales	Documentalista

Docente: Gleiston Cicerón Guerrero Ulloa

Período académico 2026

Corte documental: 1 de septiembre de 2026

FECHA PERMITIDO DE ÚLTIMO COMMIT:

Martes 1 de septiembre del 2026 hasta Hora: 18h00

<https://github.com/JoseLozanoMorales/TiendaTech>

Índice

Resumen	v
Abstract	vi
1. Introducción	1
1.1. Objetivos y alcance	1
1.2. Corte de evidencia y organización	1
2. Estado del arte	2
2.1. Consistencia, orden y datos distribuidos	2
2.2. Procesamiento paralelo y verificación de consultas	2
2.3. Microservicios y clientes	2
2.4. Pruebas y entrega continua	3
2.5. Criterio bibliográfico	3
3. Diseño del sistema	3
3.1. Contexto y contenedores	3
3.2. Fronteras de dominio	4
3.3. Componentes y capas	5
3.4. Despliegue y decisiones	6
4. Propiedades distribuidas	6
4.1. Consistencia por agregado	6
4.2. Fragmentación y réplica	7
4.3. Modelo de fallos y orden	7
5. Implementación	8
5.1. Comunicación y contratos	8
5.2. Acceso a datos y control de transacciones	8
5.3. Procesamiento distribuido	8
5.4. Interfaces y capas	9
5.5. Instrumentación	9
6. Diseño del estudio	10
6.1. Preguntas	10
6.2. Factores, unidad experimental y procedimiento	10
6.3. Variables e invariantes	11
6.4. Análisis estadístico	11
6.5. Datos y paralelismo	12
7. Resultados	12
7.1. Coordinación local	12
7.2. Procesamiento analítico	14
7.3. Persistencia y tolerancia a fallos	15
8. Discusión	16
8.1. Qué permite concluir la comparación	16
8.2. Paralelismo y motor de datos	16

8.3. Decisiones de cierre	17
9. Amenazas a la validez	17
9.1. Validez interna	17
9.2. Validez externa	18
9.3. Validez de constructo	18
9.4. Validez de conclusión	18
10. Calidad del producto	18
10.1. Modelo y criterios	18
10.2. Cobertura y complejidad	19
10.3. CI, carga y observabilidad	20
11. Gestión de la revisión cruzada	20
11.1. Criterio de respuesta	20
11.2. Justificación y coste de la deuda	23
12. Conclusiones	24
12.1. Respuestas a las preguntas del banco	24
12.2. Balance acumulativo	25
13. Conclusiones individuales	25
13.1. Jhinson Stalyn Aucatoma Celorio	25
13.2. Jeremy Ruperto Gaibor Rodriguez	26
13.3. Andy Paul Sanchez Pilaloe	26
13.4. José Alejandro Lozano Morales	27
14. Reflexión ética	27
15. Declaraciones	28
15.1. Autoría y contribución	28
15.2. Uso de inteligencia artificial generativa	29
15.3. Reproducibilidad documental	30
16. Bibliografía	31

Índice de figuras

1.	C4 nivel 1: contexto del sistema implementado. Elaboración propia.	4
2.	C4 nivel 2: contenedores lógicos de aplicación. La agrupación Java resume seis servicios independientes.	4
3.	C4 nivel 3: componentes del recorrido de reserva; vista propia de responsabilidades.	5
4.	Vista de despliegue por responsabilidades. Réplica entre nodos; ensayo de clúster en un mismo equipo.	6
5.	Interfaces web conservadas de la entrega anterior.	9
6.	Interfaces Android conservadas como evidencia de implementación.	9
7.	Distribución de p95 de compra por ejecución: cinco repeticiones en cada caja; bigotes mínimo–máximo. Elaboración propia desde el CSV crudo. . .	13
8.	T1: media e intervalo del 95 % publicados, por modalidad y ejecutores. Elaboración propia.	15

Índice de tablas

1.	Fronteras y restricciones del sistema.	5
2.	Consistencia, réplica y límites por agregado.	7
3.	Protocolo solicitado y configuración realmente ejecutada.	11
4.	Latencia de compra local y confirmaciones por segundo. IC del 95 % del p95 mediano.	12
5.	Recursos y abortos de condiciones sin fallo, medianas de recursos.	14
6.	Concurrencia 400: medianas entre ejecuciones de p50, p99 y tiempo hasta compensación, en ms.	14
7.	Tiempo de T1 por motor y modalidad.	15
8.	Tiempos registrados en comparación de planes, normalizados a milisegundos.	16
9.	Evaluación por características ISO/IEC 25010:2023.	19
10.	Cobertura en el alcance de los informes disponibles.	19
11.	Respuesta y evidencia individual de revisión cruzada.	21
12.	Deuda aceptada para continuidad y coste orientativo de resolución.	24
13.	Contribuciones por rol, sujetas a revisión final de sus autores.	29

Resumen

Objetivo: consolidar la evolución de TiendaTech y evaluar qué propiedades de su arquitectura distribuida quedan respaldadas por evidencia. **Método:** revisión del repositorio y de las incidencias, análisis de mediciones de CockroachDB y Spark, y síntesis de 120 ejecuciones de un laboratorio de coordinación con dos estrategias, cuatro concurrencias y tres condiciones de fallo. **Resultados:** el laboratorio registró cero violaciones en sus cuatro comprobaciones; sin fallos y con concurrencia 400, la mediana del percentil 95 fue 3401,944 ms para la implementación denominada 2PC y 24712,718 ms para Saga. Ambas comparten SQLite y un bloqueo global, por lo que la comparación no demuestra el comportamiento de coordinadores distribuidos reales. Spark sin particionamiento JDBC no aceleró T1 al pasar de uno a cuatro ejecutores. Los informes de calidad registran complejidades máximas inferiores a diez y coberturas superiores al 70 % en el alcance instrumentado, no en todos los componentes. La revisión cruzada conserva diez incidencias abiertas. **Contribución:** una memoria trazable que separa implementación, medición y limitaciones, e identifica los controles necesarios para extender los resultados a producción. No se acreditan una hora de disponibilidad, una comparación reglas–RAG ni una medición final de carga del sistema desplegado.

Palabras clave: sistemas distribuidos, coordinación, CockroachDB, Spark, calidad, reproducibilidad.

Abstract

Objective: to consolidate TiendaTech’s evolution and determine which distributed-system properties are supported by evidence. **Method:** repository and issue review, analysis of CockroachDB and Spark measurements, and synthesis of 120 coordination-laboratory runs covering two strategies, four concurrency levels and three fault conditions. **Results:** the laboratory reported no violations of its four checks. At concurrency 400 without faults, median run-level p95 latency was 3401.944 ms for the implementation named 2PC and 24712.718 ms for Saga. Both share SQLite and a global lock; therefore, this comparison does not demonstrate the behavior of real distributed coordinators. Spark without JDBC partitioning did not accelerate T1 when increasing from one to four executors. Quality reports show maximum method complexity below ten and coverage above 70 percent within the instrumented scope, not across every component. Ten cross-review issues remain open. **Contribution:** a traceable cumulative report separating implementation, measurement and limitations, and identifying the controls required before extrapolating to production. One-hour availability, a rules-versus-RAG comparison and a final deployed-system load measurement are not established.

Keywords: distributed systems, coordination, CockroachDB, Spark, quality, reproducibility.

1. Introducción

TiendaTech es un proyecto académico de comercio electrónico de componentes de computación. Integra clientes web y móvil, autenticación, catálogo, inventario, pedidos, ventas, compras a proveedores y un asistente de armado. El problema técnico consiste en conservar reglas de negocio cuando la información cruza procesos y se presentan reintentos, fallos de comunicación o indisponibilidad de datos. Una compra no debe confirmar un pago distinto del esperado, descontar inventario indebidamente ni permanecer en un estado parcial sin una política de recuperación.

La cuantificación disponible corresponde al entorno experimental, no a pérdidas comerciales de una empresa. El conjunto analítico histórico incluye 600 000 pedidos y detalles, 10 000 usuarios y 570 000 filas tras el filtrado y unión. El ensayo de coordinación comprende 24 condiciones y cinco repeticiones por condición, con concurrencias nominales entre 50 y 400. Estos tamaños delimitan la evaluación: no se dispone de una línea base de incidentes ni de clientes reales que permita estimar impacto económico. La ausencia de esos datos no se reemplaza por cifras supuestas.

1.1. Objetivos y alcance

El objetivo general es documentar y evaluar una implementación distribuida reproducible, identificando sus garantías y límites. Los objetivos específicos son: describir las fronteras y decisiones arquitectónicas; relacionar consistencia, replicación y fallos con agregados del negocio; presentar mediciones de procesamiento y coordinación; contrastar calidad y revisión cruzada con evidencia; y responder las preguntas del banco experimental sin extrapolar más allá de su implementación.

La memoria acumula E1 (propuesta y decisiones), E2 (modelo y distribución de datos), E3 (integración y análisis) y E4 (refactor, seguridad, pruebas y evaluación). El nombre anterior del repositorio fue **PFC-AppsDistribuidas**; la denominación adoptada es **TiendaTech**. Esta aclaración conserva la interpretación de rutas y mensajes históricos.

1.2. Corte de evidencia y organización

El corte de código publicado es **6cfe0e8**, identificado por su hash completo en los enlaces de evidencia. La carpeta local conserva modificaciones no integradas y no se considera idéntica a esa revisión. Los resultados presentados proceden de artefactos existentes: durante el cierre documental no se ejecutaron nuevas pruebas de carga ni se modificó producción. El cierre de esta memoria no equivale a declarar aprobados todos los pasos técnicos.

El texto avanza desde el estado del arte y el diseño hasta la implementación; define después el estudio, presenta resultados separados de su discusión y examina amenazas, calidad y revisión cruzada. Las conclusiones responden las preguntas experimentales; las reflexiones individuales, ética y declaraciones explicitan la responsabilidad del equipo. La bibliografía sigue estilo IEEE con Biber. Los identificadores de evidencias permiten localizar el dato original y diferenciarlo de su interpretación.

2. Estado del arte

2.1. Consistencia, orden y datos distribuidos

Özsu y Valduriez [1] proporcionan el marco de fragmentación, replicación y procesamiento distribuido. En TiendaTech se usa para distinguir un esquema lógico de una distribución física: crear esquemas por servicio no demuestra que los datos estén fragmentados entre nodos. Taft et al. [2] describen CockroachDB como SQL distribuido resiliente; su pertinencia está en conectar transacciones y réplica, no en trasladar resultados de esa publicación a un clúster académico.

Gilbert y Lynch [3] formalizan el conflicto entre consistencia y disponibilidad bajo particiones. Brewer [4] revisita su interpretación y evita tratar CAP como una elección estática de dos letras para todo el producto. Abadi [5] añade que también existen compromisos de latencia y consistencia durante el funcionamiento normal. En consecuencia, esta memoria separa la escritura de inventario, que requiere control de concurrencia, de la lectura de catálogo y de los resultados analíticos que admiten desfase.

Lamport [6] fundamenta el orden causal mediante relojes lógicos. Su aplicación al intercambio de reservas permite ordenar eventos relacionados, pero no convierte un timestamp en un mecanismo de exclusión mutua ni en consenso. Un reloj creciente tampoco evita por sí solo que el receptor aplique dos veces una operación. Esta distinción explica por qué la idempotencia de extremo a extremo permanece como deuda separada.

2.2. Procesamiento paralelo y verificación de consultas

Amdahl [7] establece un límite para acelerar una carga fija cuando existe una parte serial. Gustafson [8] cambia el planteamiento al considerar cargas escaladas. No son predicciones intercambiables: los CSV del proyecto mantienen el conjunto de datos y por ello el análisis principal es de escalabilidad fuerte. Si Spark tarda más con más ejecutores, aumentar artificialmente la carga para obtener otra conclusión alteraría la pregunta inicial.

Rigger y Su [9] proponen particionar consultas para encontrar errores lógicos en motores de bases de datos. Esta partición de predicados no equivale a fragmentar físicamente tablas ni a dividir lecturas JDBC. Su aporte metodológico al proyecto es la necesidad de comprobar resultados equivalentes al comparar planes, no asumir que una consulta rápida es correcta. Los conteos y agregados deben conservarse antes de atribuir una ventaja al tiempo de ejecución.

2.3. Microservicios y clientes

Márquez et al. [10] revisan patrones y tácticas de microservicios; Zhong et al. [11] investigan diseño guiado por dominio. Ambas perspectivas orientan a justificar límites por responsabilidad y dependencia, no solo por el número de carpetas o contenedores. Para TiendaTech, pedidos y ventas tienen fronteras conceptuales distintas, aunque compartir una base física conserve acoplamientos operativos.

Nawrocki et al. [12] comparan enfoques nativos y multiplataforma para aplicaciones móviles.

El proyecto utiliza un cliente Android y debe juzgar esa decisión según integración con el dispositivo, mantenibilidad y pruebas, sin atribuirle superioridad universal. Las capturas muestran interfaces, mientras que una garantía de compatibilidad requiere ejecutar sobre dispositivos y versiones concretos.

2.4. Pruebas y entrega continua

Shahin et al. [13] distinguen integración, entrega y despliegue continuos. La distinción impide presentar cualquier workflow exitoso como una publicación segura del producto. Rostami Mazrae et al. [14] estudian uso y migración de herramientas CI/CD; para el proyecto, el valor está en conservar controles y artefactos verificables, no en acumular proveedores de automatización.

Hui et al. [15] sintetizan desafíos y brechas de prueba de microservicios. Este marco explica por qué una alta cobertura unitaria del dominio no reemplaza contratos entre servicios, pruebas de recuperación o recorridos de usuario. La síntesis de estas quince fuentes conduce a un criterio transversal: una decisión debe tener una justificación y una evidencia del mismo nivel de abstracción. Ni una prueba local demuestra disponibilidad distribuida ni una captura de pantalla demuestra corrección transaccional.

2.5. Criterio bibliográfico

Las quince fuentes académicas anteriores disponen de DOI. Las normas ISO y el código de ética se citan adicionalmente por su identificador y sitio institucional, sin inventar un DOI. Por tanto, se cumple el mínimo de literatura académica con DOI, pero la lectura literal de que *toda* referencia debe tenerlo exige una excepción para estas fuentes normativas solicitadas por la propia guía. La verificación bibliográfica identifica el trabajo; no implica que se haya reproducido su experimento.

3. Diseño del sistema

3.1. Contexto y contenedores

Las figuras 1 y 2 son vistas propias, reconstruidas para este corte. La primera delimita el sistema respecto de sus usuarios; la segunda representa unidades desplegables y sus dependencias. Se evita representar una pasarela bancaria real como integrada: los pagos del banco experimental son simulados.

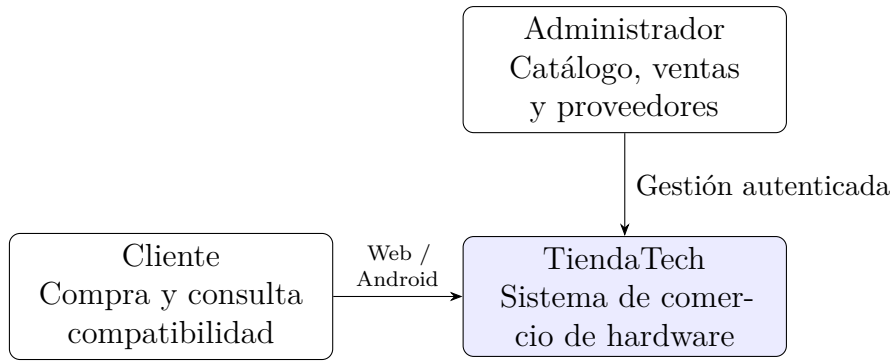


Figura 1: C4 nivel 1: contexto del sistema implementado. Elaboración propia.

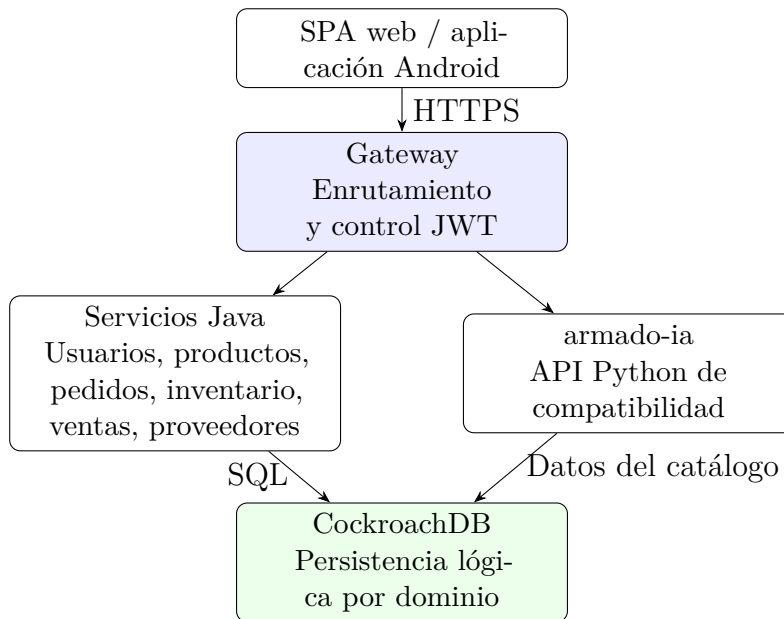


Figura 2: C4 nivel 2: contenedores lógicos de aplicación. La agrupación Java resume seis servicios independientes.

3.2. Fronteras de dominio

La tabla 1 describe la responsabilidad principal. Los nombres cortos son lógicos; no sustituyen los nombres concretos de Compose. Separar responsabilidades no garantiza aislamiento físico, puesto que las conexiones dependen del clúster compartido. Un cambio de clave en pedidos puede propagarse a consumidores aunque cada servicio tenga su propio paquete.

Tabla 1: Fronteras y restricciones del sistema.

Servicio	Responsabilidad	Restricción relevante
Usuarios	Identidad y autenticación	No publicar claves ni tokens en evidencia.
Productos	Catálogo, precios y medios	No es propietario del saldo de inventario.
Inventario	Stock y reservas	Debe impedir descuentos duplicados y negativos.
Pedidos	Carrito y órdenes	Coordina solicitudes; no equivale a transacción global.
Ventas	Registro comercial y facturación	Debe preservar importes y referencias.
Órdenes a proveedores	Abastecimiento	No confundir compra al proveedor con venta al cliente.
armado-ia	Reglas y asistencia de armado	Resultado sujeto a catálogo y reglas disponibles.
Gateway	Acceso y rutas	Es control de entrada, no regla de negocio.

3.3. Componentes y capas

La figura 3 presenta la reserva de stock como una vista de componentes; no pretende mostrar todas las clases del sistema. El contrato compartido define el mensaje mientras que aplicación y dominio conservan las reglas. El refactor Java organiza **presentation**, **application**, **domain** e **infrastructure**. En Python se documenta la correspondencia de responsabilidades con sus convenciones de módulos, sin afirmar que todos los directorios tengan los mismos nombres.

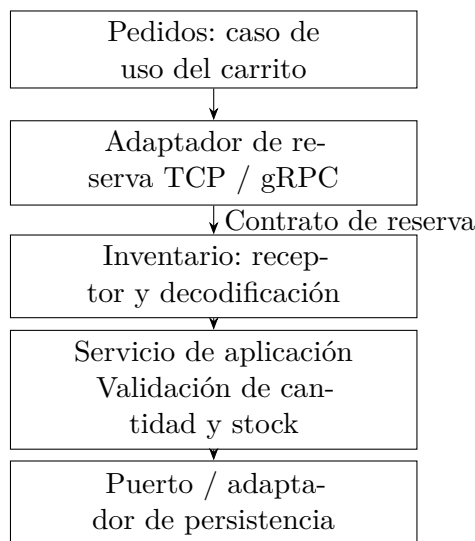


Figura 3: C4 nivel 3: componentes del recorrido de reserva; vista propia de responsabilidades.

3.4. Despliegue y decisiones

La figura 4 separa aplicación, persistencia y observabilidad. Los tres nodos del ensayo local no son tres centros de datos ni tres dominios de fallo independientes. La conexión segura del despliegue no debe confundirse con el entorno de pruebas de integración, que puede usar un único contenedor temporal.

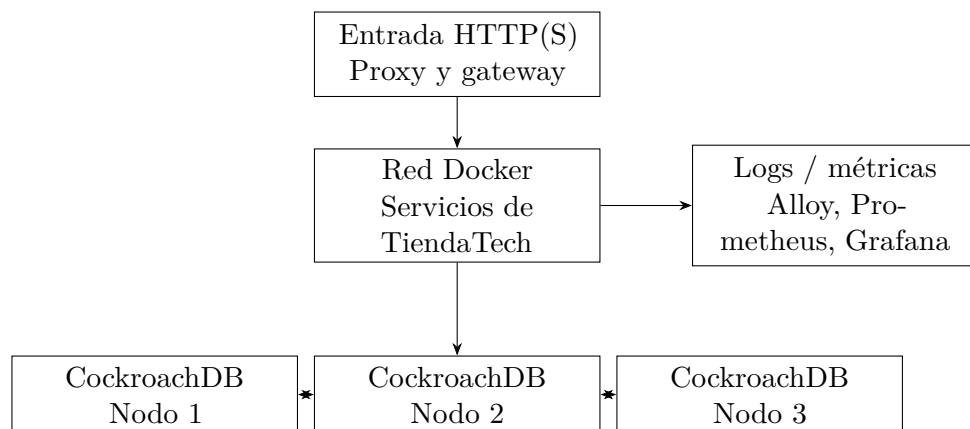


Figura 4: Vista de despliegue por responsabilidades. Réplica entre nodos; ensayo de clúster en un mismo equipo.

Los [ADR versionados](#) conservan contexto, decisión y consecuencias. ADR-003 explica la fragmentación temporal de pedidos; ADR-004 aborda Raft; ADR-005 registra patrones de diseño; ADR-007 centraliza JWT; ADR-008 describe capas Python y el límite de confianza de armado-ia. ADR-009 recupera la decisión de microservicios, ADR-010 el gateway y ADR-011 el motor de compatibilidad. ADR-012 fragmenta el índice del catálogo por categoría sin cambiar su clave primaria. Las decisiones móviles están en los registros 001 y 002.

No todas las consecuencias previstas en E1 se convirtieron en resultados medidos. Escalar un servicio por separado es una posibilidad arquitectónica; demostrar que mejora latencia o disponibilidad requiere un experimento. Asimismo, un ADR que justifica JWT en el gateway no hace seguro un servicio accesible directamente desde redes no confiables. Su validez depende de la exposición real y de pruebas negativas de acceso.

4. Propiedades distribuidas

4.1. Consistencia por agregado

La tabla 2 formula la política requerida y su límite de evidencia. CAP se interpreta ante particiones de red: la réplica por quórum puede rechazar o demorar escrituras cuando no dispone de mayoría. No se promete disponibilidad total junto con consistencia fuerte durante cualquier partición [3], [4], [5].

Tabla 2: Consistencia, réplica y límites por agregado.

Agregado	Política	Límite
Inventario	Serializar cambios y comprobar stock; réplica del motor.	Atomicidad local no prueba idempotencia entre servicios.
Pedido y pago	Confirmar solo con condiciones satisfechas; compensar fallo.	El laboratorio no implementa commit distribuido real.
Usuarios	Preservar identidad y validar autorización.	JWT no replica estado ni revoca instantáneamente por sí solo.
Catálogo	Lectura consistente según consulta; réplica física.	No se midió un contrato de frescura de caché.
Analítica	Instantánea o extracción identificable.	Resultados derivados pueden quedar desfasados respecto de ventas.

4.2. Fragmentación y réplica

Pedidos utiliza límites temporales de rangos; el catálogo divide el índice secundario por categoría. ADR-012 documenta un conjunto sintético de 150 productos y diez rangos, con réplicas votantes en los nodos 1, 2 y 3. Es fragmentación del índice, no una demostración de partición exclusiva de todo el agregado por nodo. El identificador del producto se mantiene para no romper referencias desde carrito, medios e inventario.

La tolerancia depende de la mayoría disponible y del tipo de fallo. Perder un nodo de tres permite conservar mayoría en la configuración ensayada, pero una caída del equipo anfitrión puede afectar los tres simultáneamente. La replicación tampoco reemplaza copias de seguridad: una modificación lógica incorrecta puede replicarse. La evidencia de reintegración se presenta en resultados sin convertirla en un porcentaje de disponibilidad anual.

4.3. Modelo de fallos y orden

Se consideran omisión de respuesta, demora, caída de proceso y reintentos. El laboratorio del Paso 8 representa omisión y demora mediante excepciones locales; no corta paquetes en una red real. No se simulon nodos maliciosos, pérdida total del anfitrión, corrupción persistente ni recuperación de un coordinador tras reiniciar durante un commit.

El reloj lógico cumple la regla de recepción

$$L_{\text{nuevo}} = \text{máx}(L_{\text{local}}, L_{\text{recibido}}) + 1. \quad (1)$$

La ecuación 1 establece un orden compatible con causalidad [6]; no mide duración física. Para medir latencia se emplea un reloj monotónico local. La deduplicación requiere un identificador de operación persistente y una política de reintento, además del reloj.

5. Implementación

5.1. Comunicación y contratos

La reserva TCP utiliza longitud de cuatro bytes en orden de red seguida por el JSON. Tanto emisor como receptor deben completar la lectura; una sola llamada no garantiza recibir un mensaje íntegro. La variante gRPC define el contrato en [stock_reservation.proto](#). Los stubs deben regenerarse durante la compilación.

La disponibilidad de ambas rutas no demuestra que una sea más rápida. Los ficheros del experimento TCP/gRPC deben contener observaciones y no solo cabeceras antes de usar sus percentiles. El banco de coordinación es un experimento distinto y no completa esa comparación de transportes.

5.2. Acceso a datos y control de transacciones

El extracto 1 procede del laboratorio, no del servicio productivo. Hace visible la serialización local que condiciona los resultados. Saga también conserva el mismo bloqueo global durante su recorrido; las diferencias incluyen cantidad de operaciones SQL y commits locales.

```
def two_phase_commit(self, tx, case, started):
    with self.db_lock, closing(self.connect()) as db:
        db.execute("BEGIN IMMEDIATE")
        try:
            # Líneas omitidas: stock, votos e inyector.
            ...
            db.commit()
        except Exception:
            db.rollback()
            raise
```

Listing 1: Extracto de E-2PC local: bloqueo compartido y transacción SQLite.

Los puntos suspensivos identifican líneas omitidas; el código completo está en [coordinacion_lab.py](#). No existen en este recorrido tres gestores transaccionales independientes ni recuperación durable del coordinador comparable a un protocolo 2PC distribuido.

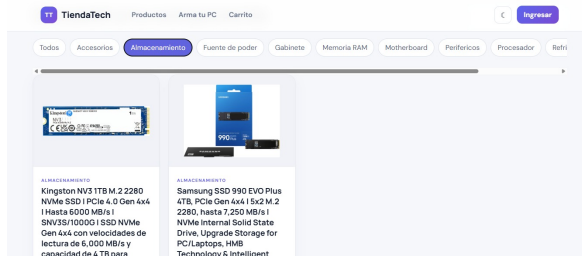
5.3. Procesamiento distribuido

El pipeline analítico calcula transformaciones equivalentes con Pandas y PySpark. La comparación conserva tarea, tamaño y resultado lógico. La lectura JDBC sin particiones limita el paralelismo de entrada; añadir ejecutores no garantiza aprovecharlos. Una variante particionada se conserva como ensayo diferente, con tres repeticiones, y no se mezcla con las cinco del escenario inicial.

La fragmentación de CockroachDB organiza rangos del motor; la partición JDBC determina cómo Spark solicita datos en paralelo. Los tiempos incluyen planificación, comunicación y materialización. Para reproducir se deben conservar consulta, recursos, número de ejecutores y criterio de finalización.

5.4. Interfaces y capas

Las figuras 5 y 6 conservan evidencia visual de E4. No son pruebas de aceptación nuevas ni mediciones de disponibilidad. La SPA permite consultar catálogo y armado; el panel administrativo organiza usuarios, productos y operaciones comerciales. Android ofrece otro punto de entrada a los servicios.



(a) Catálogo.

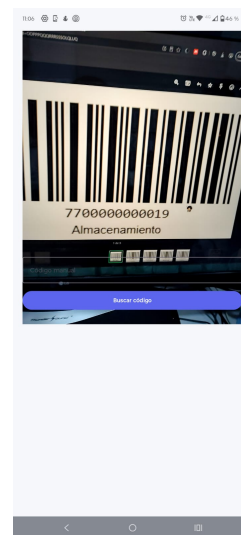


(b) Armado asistido.

Figura 5: Interfaces web conservadas de la entrega anterior.



(a) Vista inicial.



(b) Navegación.

Figura 6: Interfaces Android conservadas como evidencia de implementación.

La separación de capas permite probar reglas sin desplegar la interfaz ni el motor de datos. Sin embargo, excluir adaptadores del informe de cobertura cambia el denominador y debe declararse. Persisten observaciones sobre DTO y reutilización de lógica de órdenes; el refactor no ha eliminado todo acoplamiento.

5.5. Instrumentación

El extracto 2 identifica las unidades medidas. La memoria no es el consumo residente del contenedor, y los segundos de CPU no son un porcentaje.


```

tracemalloc.start()
cpu_0 = time.process_time()
t0 = time.perf_counter()
with ThreadPoolExecutor(max_workers=conurrencia) as pool:
    rows = list(pool.map(lab.purchase, cases))
duracion = time.perf_counter() - t0
cpu = time.process_time() - cpu_0
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

```

Listing 2: Instrumentación del ejecutor del Paso 8.

La reproducción debe partir del hash indicado, no de una mezcla de archivos locales y remotos. Compilar esta memoria no ejecuta experimentos ni modifica la base.

6. Diseño del estudio

6.1. Preguntas

Se formulan cinco preguntas operativas para responder los requisitos del banco del Paso 7:

1. ¿Las estrategias conservan pago exacto, descuento único, stock no negativo y compensación completa en los casos observados?
2. ¿Cómo varían latencia y confirmaciones por segundo entre E-2PC y E-Saga, concurrencias y modos de fallo?
3. ¿Qué recuperación puede observarse al fallar la pasarela y qué queda fuera del modelo?
4. ¿El banco determinista de compatibilidad valida el asistente real y permite comparar reglas con RAG?
5. ¿Los instrumentos permiten reproducir el experimento distribuido exigido o solamente una aproximación local?

Se añaden dos estudios acumulados: escalabilidad analítica y comportamiento del clúster. No se combinan estadísticamente con el laboratorio de coordinación.

6.2. Factores, unidad experimental y procedimiento

La matriz contiene dos estrategias, concurrencias 50, 100, 200 y 400, y modos **none**, **omission**, **timing**. Cada una de las 24 condiciones tiene cinco repeticiones. La unidad resumida es la ejecución; cada ejecución lanza un lote finito de tantas compras como concurrencia nominal. No es una carga sostenida de usuarios navegando por el sistema.

El ejecutor reinicia una base por repetición, establece stock inicial $3c + 10$, genera casos y obtiene el reporte del oráculo. La semilla base es 2026, pero su cálculo incorpora la longitud del nombre de estrategia y modo; las estrategias no reciben necesariamente el mismo conjunto de fallos. La comparación no es un diseño emparejado perfecto.

Tabla 3: Protocolo solicitado y configuración realmente ejecutada.

Aspecto	Referencia de la guía	Evidencia del corte
Repeticiones	Cinco por condición	Cinco; 120 ejecuciones.
Retardo	Cinco segundos	0,05 segundos.
Calentamiento	Sesenta segundos	Cero; el parámetro solo espera, no genera tráfico.
Duración	Corridas sostenidas sugeridas	Lote finito de 50–400 operaciones.
Pasarela	Intermediario y fallo de respuesta	Función local con excepción.
Coordinación	Participantes distribuidos	Una SQLite y bloqueo global.
CPU y memoria	Porcentaje y MiB operativos	Segundos CPU y asignaciones Python.

La tabla 3 distingue parametrización de ejecución. Cambiar demora y espera no transforma el simulador en un ensayo distribuido ni implementa calentamiento bajo carga.

6.3. Variables e invariantes

Se miden percentiles de duración de compra, confirmaciones por segundo, abortos, violaciones del oráculo, tiempo hasta compensación, CPU y memoria rastreada. Aunque algunas columnas se denominan `latencia_pago`, el código obtiene el tiempo de `lab.purchase`; se interpreta como compra local y no como pago aislado.

El oráculo consulta pagos de órdenes confirmadas, suma de movimientos, mínimos históricos y restauración de órdenes compensadas. La comprobación de descuento único verifica un saldo neto igual a la cantidad negativa, no un único movimiento: duplicados que se cancelen podrían pasar. Tampoco enumera exhaustivamente estados pendientes u operaciones desaparecidas tras rollback. Es una verificación más débil que una prueba completa de las invariantes en producción.

$$R = \frac{N_{\text{confirmadas}}}{t_{\text{fin}} - t_{\text{inicio}}}, \quad r_a = \frac{N_{\text{abortadas}}}{N_{\text{operaciones}}}. \quad (2)$$

La ecuación 2 explicita denominadores. Sumar violaciones de distintas consultas y dividir por operaciones no siempre produce una proporción binomial: una operación podría violar más de una regla. Los intervalos publicados de inconsistencia se interpretan con esa reserva.

6.4. Análisis estadístico

Se calcula mediana de los cinco p95 e intervalo bootstrap percentil del 95 % con mil remuestreos. No es el p95 de todas las operaciones mezcladas. Para abortos y compatibilidad se emplea Wilson; para latencias se publican Mann–Whitney U aproximado y tamaño de efecto:

$$\hat{A}_{12} = \frac{\sum_{i=1}^{n_1} \sum_{j=1}^{n_2} [\mathbb{I}(x_i > y_j) + \frac{1}{2}\mathbb{I}(x_i = y_j)]}{n_1 n_2}. \quad (3)$$

En 3, x es 2PC y y es Saga. Cero significa que ningún p95 observado de 2PC supera al de Saga; no implica certeza poblacional. Se conservan doce comparaciones sin ajuste múltiple, con cinco observaciones por grupo. Los valores p son aproximaciones normales, no pruebas exactas; el script no incorpora todas las correcciones de empates o continuidad. La inferencia es exploratoria y el orden fijo de ejecución limita una atribución causal.

6.5. Datos y paralelismo

Para Spark se comparan tiempos de la misma tarea y lectura:

$$S(N) = \frac{T(1)}{T(N)}, \quad E(N) = \frac{S(N)}{N}, \quad S_{\text{Amdahl}}(N) = \frac{1}{(1-f) + f/N}. \quad (4)$$

La ecuación 4 usa f como fracción paralelizable [7]. Despejar la parte no escalable $q = (1/S - 1/N)/(1 - 1/N)$ puede producir valores superiores a uno cuando hay desaceleración: no son fracciones físicas, sino señal de sobrecostos fuera del modelo simple.

Los ensayos del clúster comparan planes y una secuencia de caída/reintegración, sin repeticiones suficientes para estimar una mejora general. Se conservan observaciones individuales, convirtiendo unidades explícitamente.

7. Resultados

7.1. Coordinación local

La tabla 4 transcribe las 24 condiciones de [experimento_resumen.csv](#). Cada fila resume cinco ejecuciones. La figura 7 representa los p95 por ejecución sin mezclar niveles de concurrencia. No se excluyeron ejecuciones al generar la figura.

Tabla 4: Latencia de compra local y confirmaciones por segundo. IC del 95 % del p95 mediano.

Estrategia	Conc.	Fallo	p95 (ms)	IC (ms)	Órdenes/s
2PC	50	none	371.9	[364.1; 374.0]	123.12
2PC	50	omission	346.7	[331.2; 372.9]	114.57
2PC	50	timing	783.7	[524.4; 876.2]	50.42
2PC	100	none	773.1	[646.7; 988.6]	111.33
2PC	100	omission	714.3	[685.9; 795.3]	112.98
2PC	100	timing	1199.8	[929.7; 1347.3]	69.63
2PC	200	none	1656.8	[1507.7; 1864.5]	108.61
2PC	200	omission	1447.0	[1406.0; 1626.1]	113.99
2PC	200	timing	2621.2	[2288.6; 2771.6]	64.61
2PC	400	none	3401.9	[3158.9; 3636.8]	108.07
2PC	400	omission	3085.3	[2927.9; 3139.0]	107.05
2PC	400	timing	5311.8	[4791.1; 5706.5]	61.14

Estrategia	Conc.	Fallo	p95 (ms)	IC (ms)	Órdenes/s
SAGA	50	none	2017.0	[1876.5; 2175.6]	23.38
SAGA	50	omission	1871.5	[1860.4; 1976.3]	22.26
SAGA	50	timing	2286.0	[2211.4; 2402.9]	18.31
SAGA	100	none	3585.9	[3389.5; 3880.0]	26.37
SAGA	100	omission	3946.0	[3754.5; 4079.4]	20.97
SAGA	100	timing	4752.6	[4603.1; 6325.6]	17.72
SAGA	200	none	8088.5	[7075.8; 8708.8]	23.45
SAGA	200	omission	11740.4	[9903.3; 12212.3]	14.31
SAGA	200	timing	12948.6	[12370.4; 13302.2]	13.02
SAGA	400	none	24712.7	[23577.9; 27393.2]	15.32
SAGA	400	omission	22578.9	[22065.3; 23499.9]	14.97
SAGA	400	timing	25660.3	[24655.9; 26674.3]	13.14

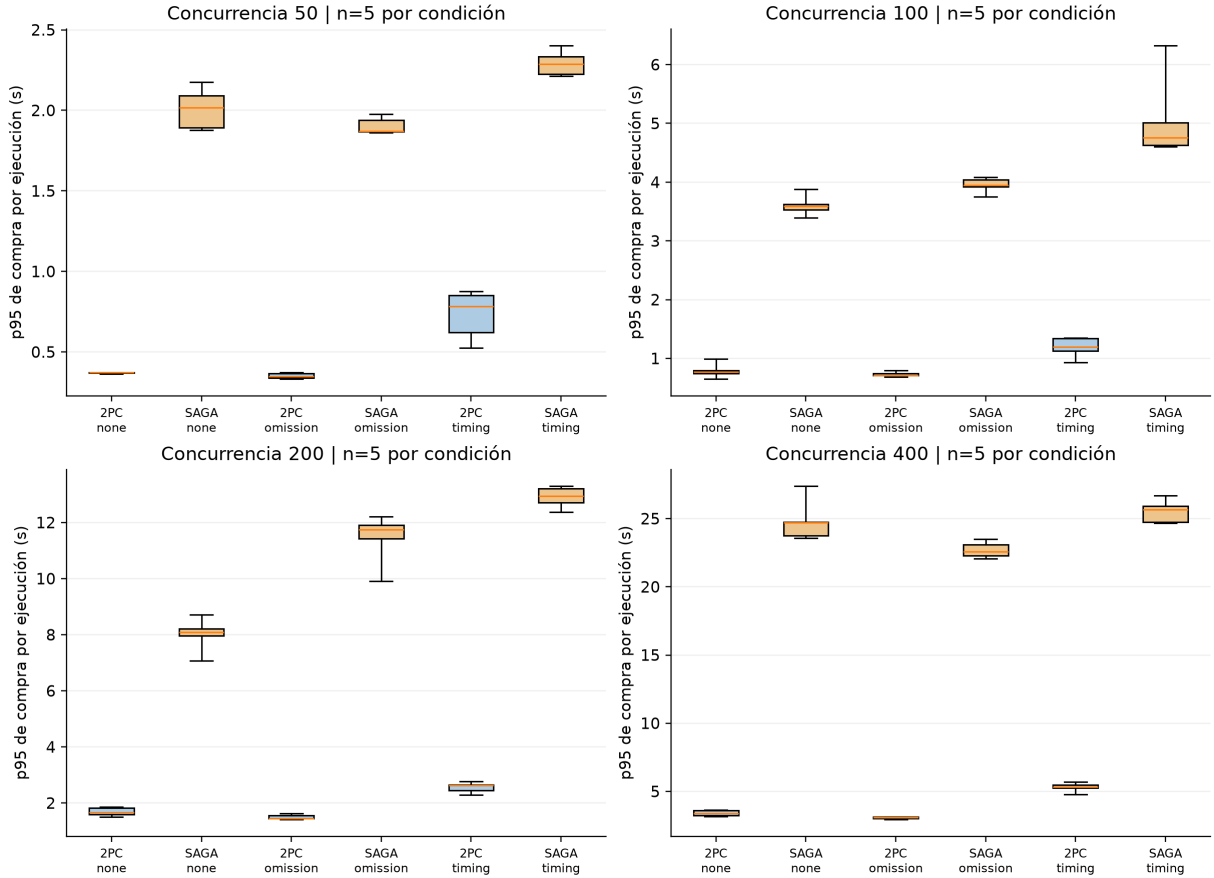


Figura 7: Distribución de p95 de compra por ejecución: cinco repeticiones en cada caja; bigotes mínimo-máximo. Elaboración propia desde el CSV crudo.

Las 120 ejecuciones registran `oracle_pass=True` y cero violaciones en las consultas del oráculo. El informe de compatibilidad registra 120 aciertos sobre 120 casos, cero falsos positivos y cero falsos negativos; el intervalo Wilson de exactitud es $[0.968981; 1]$. Esta evaluación corresponde a reglas locales de socket, RAM y potencia, tal como las implementa el evaluador del experimento.

Las doce comparaciones de p95 registran $U = 0$, $p_{\text{aprox}} = 0,009023$ y $\hat{A}_{12} = 0$. Los datos y resultados de cada contraste están en [comparaciones_mann_whitney.csv](#). La tabla 5 muestra métricas auxiliares de condiciones seleccionadas; no son medidas de recursos de producción.

Tabla 5: Recursos y abortos de condiciones sin fallo, medianas de recursos.

Estrategia	Concurrencia	CPU (s)	Memoria Python (MiB)	Tasa abortos
2PC	50	0.141	0.266	0.0
2PC	400	1.641	2.193	0.0
SAGA	50	0.750	0.266	0.0
SAGA	400	6.078	2.221	0.0

Tabla 6: Concurrencia 400: medianas entre ejecuciones de p50, p99 y tiempo hasta compensación, en ms.

Estrategia	Fallo	p50	p99	Hasta compensación
2PC	none	1799,648	3534,757	—
2PC	omission	1557,841	3252,304	—
2PC	timing	3001,869	5633,613	—
Saga	none	12630,484	25661,211	—
Saga	omission	12141,354	23441,936	21795,425
Saga	timing	13085,967	26835,680	23846,341

La tabla 6 se calcula desde el CSV crudo. El guion indica que no hay evento de compensación; el script guarda cero en esos casos, que no se interpreta como recuperación instantánea. La última columna resume el p95 del tiempo acumulado hasta el evento, calculado en cada ejecución.

7.2. Procesamiento analítico

La tabla 7 recoge medias de T1 en [tiempos_resumen.csv](#). La lectura particionada tiene tres repeticiones y las demás cinco. La figura 8 visualiza esas observaciones agregadas, no una simulación.

Tabla 7: Tiempo de T1 por motor y modalidad.

Motor / lectura	Ejecutores	Repeticiones	Media (s)
Pandas / SQL directo	—	5	2,847172
Spark / JDBC sin partición	1	5	14,557712
Spark / JDBC sin partición	2	5	17,115246
Spark / JDBC sin partición	4	5	19,132645
Spark / JDBC particionado	1	3	17,966838
Spark / JDBC particionado	2	3	16,799444
Spark / JDBC particionado	4	3	17,175005

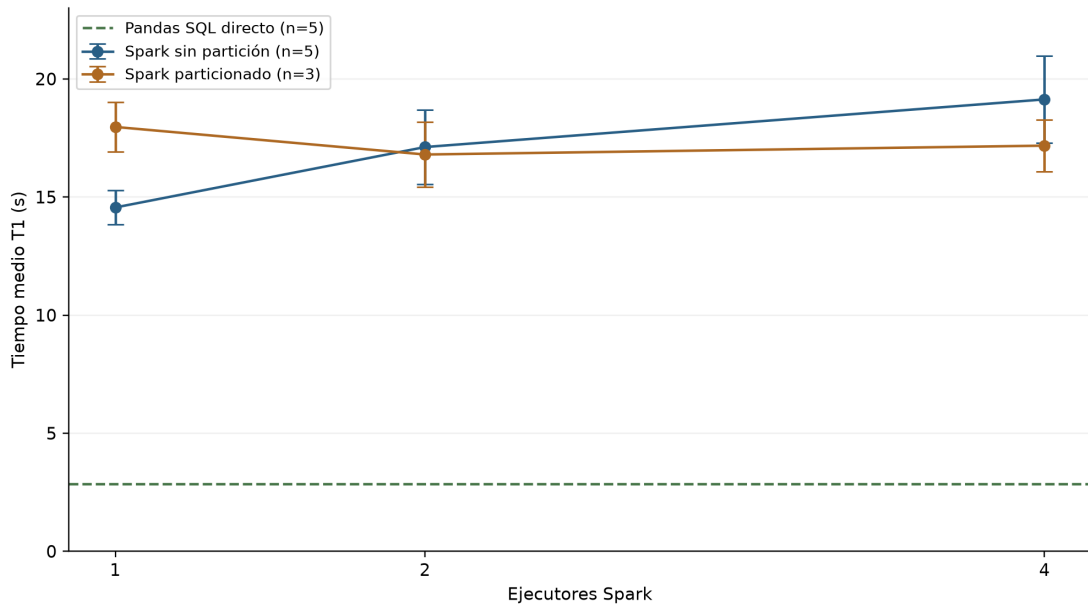


Figura 8: T1: media e intervalo del 95 % publicados, por modalidad y ejecutores. Elaboración propia.

Para Spark sin particionamiento, $S(4) = 0,760884$ y $E(4) = 0,190221$, calculados a partir de las medias. T5 registra 3,526321 s para Pandas y 33,166240, 35,589197 y 37,834008 s para Spark con uno, dos y cuatro ejecutores, respectivamente.

7.3. Persistencia y tolerancia a fallos

La comparación de planes conserva los tiempos individuales de la tabla 8. No se calculan intervalos al no disponer aquí de repeticiones homogéneas de esas consultas.

Tabla 8: Tiempos registrados en comparación de planes, normalizados a milisegundos.

Consulta	Clúster 3 nodos	Nodo único
Historial de usuario	3	3
Top 10 trimestral	625	1100
Cabecera y detalle	11	78
Catálogo	44	2000
Reporte trimestral	4000	8000

En el ensayo de fallo, las consultas tardaron 464,01 ms antes de la caída, 4919,55 ms inmediatamente después, 428,94 ms después de treinta segundos y 918,4 ms tras la reintegración. Las cuatro conservaron el resultado 1644 y estado satisfactorio. El tiempo registrado de reintegración fue 3,62 s. Las rutas de los ficheros originales se conservan en el inventario de cierre.

8. Discusión

8.1. Qué permite concluir la comparación

En las condiciones observadas, la implementación E-2PC presenta p95 menores que E-Saga. No se concluye que el protocolo 2PC sea generalmente superior: ambas rutas están serializadas por un bloqueo y se ejecutan en un motor local. E-Saga realiza más confirmaciones individuales; el coste de esas operaciones puede dominar la diferencia. Para evaluar coordinación distribuida habría que separar participantes, persistir decisiones y estudiar recuperación de procesos, incertidumbre de respuesta y reintentos.

El cero de violaciones es un resultado del oráculo, no una demostración de corrección. El saldo neto de movimientos es más débil que la unicidad de descuento, y las órdenes revertidas pueden no dejar las mismas huellas que las compensadas. Se recomienda fortalecer los controles con identificadores únicos, seguimiento de todos los casos emitidos y fallos deliberados que el oráculo deba detectar. Esa recomendación no se presenta como implementada.

La compatibilidad perfecta describe un evaluador local que replica reglas usadas por el banco sintético. No invoca el servicio real de armado ni un recuperador/generador RAG. Por tanto, no puede justificar precisión del asistente ni una ventaja de IA. La validación futura debe separar generador de etiquetas y sistema evaluado, incluir incompatibilidades de catálogo y registrar respuestas reales sobre un conjunto reservado.

8.2. Paralelismo y motor de datos

La desaceleración de T1 al añadir ejecutores es compatible con sobrecostes y entrada poco paralela. Es una interpretación, no un perfil causal exhaustivo. La variante JDBC particionada cambia el acceso y muestra un patrón diferente, pero no supera automáticamente el

tiempo de Pandas. Para este tamaño y entorno, la complejidad operativa de Spark no se justifica por una mejora de T1 observada.

Aplicar Amdahl como ajuste mecánico produciría una fracción no escalable superior a uno para la serie sin partición. Eso no significa que exista más de un cien por ciento de trabajo serial: el modelo omite sobrecostos dependientes del número de ejecutores. Gustafson plantea otra pregunta para cargas crecientes [8]; no corrige retrospectivamente el resultado de carga fija.

La caída de un nodo mostró un pico de latencia con resultado conservado. Es evidencia puntual de continuidad del clúster en ese escenario, no disponibilidad sostenida. El ensayo comparte anfitrión y no cubre partición prolongada ni corrupción. Los planes con tiempos menores en clúster tampoco aíslan por sí solos diferencias de caché, índices y recursos.

8.3. Decisiones de cierre

El cierre mantiene arquitectura y evidencias existentes, y registra deuda en lugar de modificar apresuradamente el producto. Las prioridades son idempotencia y validación de contratos, luego pruebas de recuperación y una medición oficial con recursos y tráfico definidos. La revisión bibliográfica se corrigió porque un DOI existente puede apuntar a un trabajo distinto: comprobar que el enlace abre no basta; debe coincidir con título y autores.

El producto posee instrumentos de calidad y observabilidad antes ausentes. Sin embargo, instalar Prometheus o aprobar CI no acredita un objetivo operacional. La memoria distingue esas capas para que el lector pueda reconocer el avance sin asumir garantías todavía no medidas.

9. Amenazas a la validez

9.1. Validez interna

Contención del anfitrión. Todas las ejecuciones comparten máquina y pueden sufrir carga externa. Mitigación: conservar repeticiones y recursos, aislar futuras corridas y aleatorizar orden. No se supone aislamiento retroactivo.

Serialización e implementación. SQLite y el bloqueo común confunden estrategia con coste de commits. Mitigación: restringir conclusiones al laboratorio y repetir con participantes separados antes de elegir protocolo productivo.

Semillas y orden. El cálculo de semillas varía entre estrategias y las condiciones se recorren en orden fijo. Mitigación: utilizar listas idénticas de operaciones/fallos y orden aleatorio balanceado en una réplica. Las comparaciones actuales son no emparejadas.

Configuración abreviada. Demora de 0,05 s y calentamiento cero difieren de la guía. Mitigación: publicar metadatos y no afirmar cumplimiento de las condiciones de cinco segundos y sesenta segundos; la futura espera deberá generar carga efectiva.

9.2. Validez externa

Carga sintética concentrada. Compras sobre un producto no representan navegación, usuarios heterogéneos ni distribución real de demanda. Mitigación: añadir perfiles mixtos y más productos con tamaños declarados; limitar el alcance actual a contención local.

Topología y pasarela. Un anfitrión y una función de pago no modelan latencia geográfica ni fallos bancarios. Mitigación: ensayar red y participantes independientes sin usar credenciales ni pagos reales; no extrapolar los percentiles a producción.

Catálogo artificial. Casos generados por reglas próximas al evaluador favorecen resultados perfectos. Mitigación: etiquetas independientes, conjunto reservado y llamadas al asistente real antes de evaluar reglas frente a RAG.

9.3. Validez de constructo

Latencia mal rotulada. Una columna de pago mide compra completa. Mitigación aplicada: nombrarla latencia de compra en el informe y conservar el nombre original en datos para trazabilidad.

Oráculo incompleto. El saldo neto no demuestra unicidad ni recuperación de todos los estados pendientes. Mitigación: declarar el alcance, usar historiales completos y pruebas que muten intencionalmente cada invariante.

Recursos parciales. CPU en segundos y memoria Python no equivalen a porcentaje CPU/RSS. Mitigación aplicada: unidades explícitas; futura instrumentación a nivel de proceso o contenedor.

9.4. Validez de conclusión

Cinco réplicas producen intervalos sensibles al ruido y doce pruebas aumentan el riesgo de falsos positivos. Mitigación: interpretar exploratoriamente, incrementar réplicas y predefinir corrección por comparaciones múltiples. No se afirma significación familiar al 5 %.

La suma de violaciones no es necesariamente binomial y un cero observado no prueba riesgo nulo. Mitigación: definir una variable binaria por operación para cada invariante y reportar su intervalo por separado. Los ensayos puntuales del clúster tampoco permiten inferir disponibilidad sostenida; se mantienen como observaciones descriptivas.

10. Calidad del producto

10.1. Modelo y criterios

ISO/IEC 25010:2023 organiza nueve características de calidad del producto; ISO/IEC 25023:2016 aporta medidas [16], [17]. Las normas no imponen los umbrales académicos de cobertura o complejidad usados aquí. Esos objetivos proceden de la guía y deben interpretarse según alcance y método.

La tabla 9 distingue evidencia favorable, parcial y ausente. No se declara certificación ISO ni conformidad integral por disponer de un CSV. La seguridad requiere pruebas y configuración coherentes, mientras que la mantenibilidad requiere que el código sea comprensible además de superar un umbral.

Tabla 9: Evaluación por características ISO/IEC 25010:2023.

Característica	Evidencia y límite
Adecuación funcional	Flujos y pruebas implementados; no aceptación completa de todos los recorridos.
Eficiencia	Mediciones de laboratorio y Spark; sin p95 oficial de producción.
Compatibilidad	Contratos HTTP/gRPC; Pact y ensayo completo entre consumidores no acreditados.
Capacidad de interacción	Interfaces web/Android; sin estudio formal de usuarios.
Fiabilidad	Ensayo de caída y comprobaciones de salud; falta hora de disponibilidad.
Seguridad	JWT, restricciones de exposición y pruebas negativas; no auditoría integral.
Mantenibilidad	Cobertura acotada y PMD menor que diez; persisten DTO y acoplamientos.
Flexibilidad	Contenedores y configuración; no ensayo multientorno exhaustivo.
Seguridad operacional	Riesgos de stock/pago identificados; no análisis completo de daño operacional.

10.2. Cobertura y complejidad

La tabla 10 procede de los informes de cobertura y sus notas de alcance. Las líneas corresponden a clases instrumentadas, en particular lógica de negocio; no incluyen necesariamente controladores, infraestructura, entidades, SPA o código generado. Un 100 % de ocho líneas no prueba todo el servicio.

Tabla 10: Cobertura en el alcance de los informes disponibles.

Módulo	Cubiertas	Medidas	Porcentaje
Inventario	8	8	100,00
Órdenes a proveedores	47	51	92,16
Pedidos	104	129	80,62
Productos	23	23	100,00
Usuarios	321	358	89,66
Ventas	27	27	100,00
armado-ia	405	540	75,00

El informe específico reciente de [complejidad](#) registra máximos por método de 7 en gateway, 8 en inventario, 9 en órdenes a proveedores, 7 en pedidos, 8 en productos, 9 en usuarios

y 7 en ventas. Sustituye en esta memoria la afirmación antigua de máximo 20, que aún aparece en resúmenes anteriores. Los siete valores son inferiores a diez; no se extiende esa medición PMD al servicio Python.

10.3. CI, carga y observabilidad

Se verificaron ejecuciones de GitHub Actions: [quality gate fallido](#) y [quality gate posterior satisfactorio](#). En el corte, [CI](#) y [quality gate](#) aparecen satisfactorios. Son ejecuciones de CI, no solicitudes de incorporación de cambios. La publicación de imágenes en Docker Hub pertenece a otro workflow; no se deduce de estos estados que toda publicación esté condicionada por el mismo control.

La prueba de carga previa registró 1911 solicitudes y 862 fallos. El diagnóstico relaciona parte del problema con respuestas de productos y con limitación de solicitudes por IP; no constituye aprobación de rendimiento. Un generador de cincuenta usuarios desde una sola IP puede activar la política de 300 solicitudes por minuto y debe distinguir 429 de errores 5xx. No se dispone de una repetición oficial posterior sobre el despliegue que permita declarar resuelto el objetivo de carga.

El repositorio contiene métricas Prometheus, logs JSON, recolección Alloy y configuración de Grafana. Esto acredita instrumentación, no una observación prolongada. No se acredita trazabilidad distribuida completa con OpenTelemetry ni una captura final del dashboard con la prueba oficial. Permanecen sin medir disponibilidad de una hora, p95 productivo y tasa final de errores; tampoco se infiere cobertura E2E o de la SPA a partir de pruebas unitarias Java.

11. Gestión de la revisión cruzada

11.1. Criterio de respuesta

La tabla 11 recoge las incidencias 16–78 recuperadas en el corte: 63 registros, de los cuales 50 figuran cerrados y 13 abiertos. Diez abiertos corresponden al trabajo pendiente identificado en el proyecto actual; otros tres, 40, 46 y 48, tienen títulos duplicados de 41, 47 y 49 y carecen de respuesta en el registro consultado. No se cierran automáticamente ni se ocultan en esta memoria.

AC significa aceptar y corregir según la respuesta publicada; AD significa aceptar y diferir en este cierre documental; R significa rebatir con justificación. El estado de GitHub y la acción documental son columnas distintas. Las respuestas enlazadas contienen la justificación y, cuando existen, referencias a commits. Un cierre administrativo no constituye por sí solo prueba de ejecución ni invalida los límites de las secciones de calidad y método.

Los abiertos con corrección parcial o pendiente se incorporan como deuda explícita; AD no significa que se haya publicado una nueva respuesta o alterado el proyecto. El registro de tres duplicados se conserva para evitar contar dos veces una misma corrección y para permitir una consolidación administrativa posterior.

Tabla 11: Respuesta y evidencia individual de revisión cruzada.

N.º	Observación	Estado	Acción	Evidencia
16	Aplicar framework para el frontend	Cerrado	AC	Respuesta 16
17	Aplicar temas de tareas formativas al proyecto	Cerrado	AC	Respuesta 17
18	Investigación de temas de la materia aplicables al proyecto	Cerrado	AC	Respuesta 18
19	Hacer que usuarios no dañe todo el proyecto	Cerrado	AC	Respuesta 19
20	Primera versión de aplicación móvil	Cerrado	AC	Respuesta 20
21	Primera versión de la IA asistente	Cerrado	AC	Respuesta 21
22	Aplicación móvil finalizada	Cerrado	AC	Respuesta 22
23	Asistente de IA finalizado	Cerrado	AC	Respuesta 23
24	Arreglar estructura del proyecto	Cerrado	AC	Respuesta 24
25	Implementar validación JWT en el API Gateway	Cerrado	AC	Respuesta 25
26	Configurar JWT_SECRET compartido entre Gateway y usuarios-service	Cerrado	AC	Respuesta 26
27	Definir rutas públicas del API Gateway	Cerrado	AC	Respuesta 27
28	Crear patrón reutilizable de validación JWT	Cerrado	AC	Respuesta 28
29	Cerrar exposición pública de los puertos 8081-8086	Cerrado	AC	Respuesta 29
30	Agregar healthcheck a usuarios-service	Cerrado	AC	Respuesta 30
31	Consolidar manejadores globales de excepciones de usuarios-service	Cerrado	AC	Respuesta 31
32	Unificar rutas de creación de usuarios	Cerrado	AC	Respuesta 32
33	Corregir respuesta HTTP al crear usuarios	Cerrado	AC	Respuesta 33
34	Documentar política común de autenticación y autorización	Cerrado	AC	Respuesta 34
35	Verificar integraciones internas con protección JWT	Cerrado	AC	Respuesta 35
36	Incorporar validación JWT en ventas-service y añadir healthcheck	Cerrado	AC	Respuesta 36
37	Configurar timeouts explícitos en InventarioClient	Cerrado	AC	Respuesta 37
38	Implementar el patrón outbox al generar una factura	Cerrado	AC	Respuesta 38
39	Guardar la factura y el evento outbox dentro de la misma transacción de base de datos	Cerrado	AC	Respuesta 39

N.º	Observación	Estado	Acción	Evidencia
40	Crear el proceso que reintente eventos pendientes y marque un evento como procesado solo después de la confirmación de inventario	Abierto	Dup.	Véase 41
41	Crear el proceso que reintente eventos pendientes y marque un evento como procesado solo después de la confirmación de inventario	Cerrado	AC	Respuesta 41
42	Añadir Circuit Breaker y reintentos seguros alrededor de la comunicación con inventario	Cerrado	AC	Respuesta 42
43	Coordinar con José la idempotencia del descuento de existencias	Abierto	AD	Respuesta 43
44	Reemplazar Map String,Integer por GenerarFacturaRequest y aplicar @Valid.	Cerrado	AC	Respuesta 44
45	Crear DTO de salida para facturas y dejar de exponer directamente la entidad JPA	Cerrado	AC	Respuesta 45
46	Incorporar validación JWT en ordenes-proveedores-service. y añadir healthcheck	Abierto	Dup.	Véase 47
47	Incorporar validación JWT en ordenes-proveedores-service y añadir healthcheck	Cerrado	AC	Respuesta 47
48	Configurar timeouts explícitos en InventarioClient	Abierto	Dup.	Véase 49
49	Configurar timeouts explícitos en InventarioClient	Cerrado	AC	Respuesta 49
50	Añadir Circuit Breaker y reintentos seguros para la comunicación con inventario	Cerrado	AC	Respuesta 50
51	Crear OrdenCompraResponseDTO y ProveedorResponseDTO	Cerrado	AC	Respuesta 51
52	Verificar que las transiciones enviar y cancelar sean idempotentes o rechacen correctamente las repeticiones	Cerrado	AC	Respuesta 52
53	Si se aprueba gRPC: implementar el cliente suscriptor de alertas de inventario y evitar órdenes duplicadas ante reconexiones o mensajes repetidos	Abierto	AD	Respuesta 53
54	Documentar por qué /enviar y /cancelar representan transiciones de negocio en lugar de un PATCH genérico	Cerrado	AC	Respuesta 54
55	Incorporar validación JWT en pedidos-service	Cerrado	AC	Respuesta 55
56	Reutilizar OrdenCompraService como punto único de lógica al recibir alertas	Abierto	AD	Respuesta 56
57	Añadir healthcheck	Cerrado	AC	Respuesta 57

N.º	Observación	Estado	Acción	Evidencia
58	Configurar connect timeout y read timeout en FacturaClient, ProductoClient y UsuarioClient	Cerrado	AC	Respuesta 58
59	Crear un ApiExceptionHandler global para respuestas de error uniformes	Cerrado	AC	Respuesta 59
60	Verificar que cualquier reintento de una operación de escritura sea idempotente o utilice una clave contra duplicados	Cerrado	AC	Respuesta 60
61	Añadir Circuit Breaker y reintentos controlados en los clientes internos donde sean seguros	Cerrado	AC	Respuesta 61
62	Cambiar operaciones void por respuestas HTTP explícitas, normalmente 204 No Content	Cerrado	AC	Respuesta 62
63	Corregir códigos HTTP de creación	Cerrado	AC	Respuesta 63
64	Reemplazar mapas y entidades expuestas por DTO	Abierto	AD	Respuesta 64
65	Añadir un estado formal del pedido solo si se implementará seguimiento en tiempo real	Cerrado	R	Respuesta 65
66	Mantener TCP/WebSocket como funcionalidad separada de las correcciones urgentes	Cerrado	AC	Respuesta 66
67	Validación JWT para productos e inventario.	Cerrado	AC	Respuesta 67
68	Healthcheck para productos e inventario.	Cerrado	AC	Respuesta 68
69	Idempotencia para inventario	Cerrado	AC	Respuesta 69
70	Respuesta ante eventos repetidos	Cerrado	AC	Respuesta 70
71	Mejora de productos para respuestas incorrectas	Cerrado	AC	Respuesta 71
72	Revisión de códigos HTTP en productos	Cerrado	AC	Respuesta 72
73	Preparación de inventario como servidor	Abierto	AD	Respuesta 73
74	Creación de contrato .proto	Abierto	AD	Respuesta 74
75	Pruebas de conexión	Abierto	AD	Respuesta 75
76	Exponer entidades	Abierto	AD	Respuesta 76
77	Revisión de rutas	Abierto	AD	Respuesta 77
78	Telemetría UDP	Abierto	AD	Respuesta 78

11.2. Justificación y coste de la deuda

La idempotencia del receptor no garantiza que ventas y proveedores propaguen una misma clave en cada reintento. Por eso el issue 43 permanece abierto aunque 69 y 70 estén cerrados. Las reservas gRPC tampoco implementan el canal de alertas solicitado por 53 y 73–75. Estos alcances distintos justifican no colapsar observaciones relacionadas como si

fueran equivalentes.

La tabla 12 aporta estimaciones documentales, no horas ejecutadas ni compromisos aceptados por los integrantes. Los costes de los grupos compartidos no deben sumarse por cada issue, porque corresponden a una misma funcionalidad.

Tabla 12: Deuda aceptada para continuidad y coste orientativo de resolución.

Issues	Motivo y coste de mantener la deuda	Estimación
43	Clave idempotente de extremo a extremo; riesgo de descuentos duplicados.	6–10 h
53, 56, 73–75	Alertas, suscripción y reconexión no completas; abastecimiento sigue manual.	20–32 h
64, 76	DTO parciales; acoplamiento y validación inconsistente de entradas/salidas.	8–12 h
77	Rutas y contratos por conciliar; riesgo de documentación divergente.	3–5 h
78	UDP opcional; canal y mantenimiento adicionales sin beneficio urgente demostrado.	8–12 h
40, 46, 48	Duplicados aparentes; falta consolidación, sin presumir trabajo técnico adicional.	Revisión

El issue 65 se rebate porque condicionaba el estado formal al seguimiento en tiempo real, que no se acredita como implementado. Esta justificación no elimina la necesidad de estados de negocio en el laboratorio; son alcances diferentes. Los costes técnicos reales deberán reestimarse al ejecutar cada cambio y sus pruebas.

12. Conclusiones

12.1. Respuestas a las preguntas del banco

Pregunta 1: invariantes. Las 120 ejecuciones superaron las cuatro consultas del oráculo. La respuesta es afirmativa solo para esas comprobaciones y casos: no acredita unicidad estricta de movimientos, recuperación de todo pendiente ni ausencia universal de inconsistencias. Se requiere un oráculo más fuerte para asegurar las invariantes completas.

Pregunta 2: rendimiento. E-2PC registró menores p95 que E-Saga en las doce comparaciones de concurrencia y fallo. A concurrencia 400 sin fallo, las medianas fueron 3401,944 y 24712,718 ms. La respuesta no establece superioridad general del protocolo: el resultado pertenece a implementaciones SQLite serializadas y a semillas no perfectamente emparejadas.

Pregunta 3: recuperación. El laboratorio modela rollback y compensación ante una excepción de autorización. No prueba respuesta perdida después de un pago externo exitoso, caída del coordinador ni recuperación durable distribuida. La convergencia registrada usa tiempo desde el inicio de la operación hasta el evento de compensación, no tiempo de reparación aislado de una pasarela real.

Pregunta 4: compatibilidad y RAG. El evaluador obtuvo 120/120 aciertos, pero ejecuta reglas locales del propio banco. La respuesta es negativa respecto de validar el asistente desplegado o comparar reglas con RAG. No se dispone de respuestas de ambos enfoques sobre el mismo conjunto independiente.

Pregunta 5: reproducibilidad distribuida. Los scripts, datos y semillas permiten examinar el laboratorio; la configuración ejecutada no satisface íntegramente demora, calentamiento, duración, independencia de participantes y recursos solicitados. La respuesta es parcial, no un cumplimiento completo del banco distribuido.

12.2. Balance acumulativo

TiendaTech pasó de la propuesta arquitectónica a una implementación con clientes, servicios, datos distribuidos e instrumentos de prueba. Se documentaron decisiones de fragmentación, capas y autenticación; se incorporaron CI, cobertura, métricas y evidencia de revisión. El avance es verificable, pero no elimina las diez incidencias funcionales abiertas ni las mediciones operativas ausentes.

El análisis de Spark no mostró aceleración de T1 sin particionamiento JDBC al aumentar ejecutores. El clúster conservó una consulta durante el fallo ensayado, con un pico de latencia. Ambas conclusiones son útiles aunque no sean una confirmación de las expectativas iniciales: permiten priorizar medición y coste frente a complejidad arquitectónica.

El Paso 13 queda atendido como consolidación documental con límites explícitos. No se declara que el proyecto cumpla todos los requisitos técnicos ni que se haya obtenido aprobación de similitud, autoría o evaluación docente. La continuidad requiere resolver la deuda registrada y sustituir cada resultado parcial por evidencia del mismo alcance.

13. Conclusiones individuales

Las siguientes reflexiones conservan y actualizan los textos individuales de la memoria anterior según las evidencias del corte. Su inclusión no representa firma ni aprobación personal nueva; cada integrante mantiene la responsabilidad de revisar su texto antes de la entrega institucional.

13.1. Jhinson Stalyn Aucatoma Celorio

La evolución de TiendaTech me permitió comprender que una aplicación distribuida no se considera terminada únicamente porque sus servicios respondan y sus contenedores puedan levantarse. La Entrega 4 hizo visible la importancia de convertir decisiones técnicas en controles repetibles: una regla de negocio debe tener pruebas, una modificación debe atravesar un pipeline y un problema operativo debe poder observarse mediante datos. El refactor por capas fue especialmente valioso porque separó responsabilidades que antes estaban mezcladas y permitió probar la lógica sin depender de la base de datos o de otros servicios. También aprendí que la cobertura es útil como señal, pero no sustituye el diseño de casos relevantes; alcanzar el setenta por ciento tiene sentido solamente si se

cubren decisiones que afectan inventario, órdenes, facturación y usuarios. La preparación de Prometheus, logs estructurados y Grafana mostró que medir un sistema requiere definir antes qué pregunta se desea responder. Finalmente, la evaluación ISO/IEC 25010 reforzó la necesidad de informar resultados honestos: una métrica pendiente debe conservarse como pendiente y no completarse con una estimación. Considero que el proyecto logró una base más mantenible y verificable, aunque todavía necesita ejecutar la observación prolongada, la carga oficial del despliegue, aunque las últimas ejecuciones de CI ya fueron verificadas. Mi principal aprendizaje es que la calidad no es una actividad final, sino una propiedad que debe incorporarse al flujo normal de desarrollo. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.2. Jeremy Ruperto Gaibor Rodriguez

Durante este proyecto confirmé que el trabajo con microservicios exige coordinar elementos que en una aplicación monolítica suelen permanecer juntos. El API Gateway, los contratos HTTP, la autenticación y la persistencia distribuida forman una cadena: un error en cualquiera de esos puntos puede afectar un flujo completo aunque cada servicio funcione por separado. Las pruebas de integración añadidas ayudan a reducir esa incertidumbre porque verifican recorridos reales hacia productos, usuarios y pedidos, mientras que las pruebas de seguridad comprueban que las rutas protegidas no queden expuestas accidentalmente. El pipeline representa otro cambio importante en la forma de trabajar. Ya no basta con que una modificación compile en la computadora de un integrante; las mismas pruebas, reglas de cobertura y construcciones deben ejecutarse en un ambiente reproducible. También comprendí el valor de hacer depender el build de los controles anteriores, pues publicar una imagen cuando existen pruebas fallidas trasladaría el riesgo hacia producción. La evaluación ISO reveló además que aprobar cobertura no implica aprobar mantenibilidad: el diagnóstico inicial de PMD encontró complejidad de 20, mientras que los informes posteriores registran máximos entre siete y nueve. Por ello, mi conclusión es que el proyecto avanzó de una demostración funcional hacia un producto con mecanismos de control, aunque su validación final depende de conservar evidencias reales, completar las mediciones operativas pendientes y mantener bajo control la complejidad después del refactor. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.3. Andy Paul Sanchez Pilaloe

El desarrollo de TiendaTech me ayudó a relacionar conceptos de arquitectura, pruebas y operación que anteriormente podía considerar independientes. La arquitectura en capas facilitó localizar la lógica de negocio y diseñar pruebas unitarias con dependencias simuladas. Esa separación también disminuye el costo de reemplazar una implementación de persistencia o comunicación, porque el dominio no necesita conocer detalles de infraes-

estructura. La parte de observabilidad resultó igualmente significativa: un contador permite conocer la demanda, un histograma permite analizar percentiles y un gauge representa valores que suben y bajan, como las conexiones activas. Estas diferencias son esenciales para construir un dashboard que sirva para tomar decisiones y no sea solamente decorativo. La instrumentación desarrollada deja logs JSON y métricas homogéneas en siete servicios, además de Prometheus y Alloy como componentes de recolección. Sin embargo, configurar herramientas no equivale a completar la observabilidad: la correlación por trazas y la captura de Grafana con datos reales continúan pendientes. El modelo ISO/IEC 25010 y los umbrales académicos aportaron criterios para expresar esa diferencia mediante evidencia. El aprendizaje más importante fue reconocer que documentar una limitación también es una práctica profesional. Un reporte técnico útil distingue con claridad lo implementado, lo ejecutado y lo todavía pendiente, permitiendo que otra persona continúe el trabajo sin depender de afirmaciones imposibles de reproducir. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

13.4. José Alejandro Lozano Morales

La construcción acumulativa del PFC permitió observar cómo una propuesta inicial se transforma mediante decisiones arquitectónicas, implementación, validación y operación. En esta última etapa entendí que la documentación debe evolucionar junto con el código. Un diagrama, una tabla o una conclusión pierde valor cuando describe un estado anterior del repositorio; por esa razón fue necesario actualizar la evidencia de pruebas, pipeline, observabilidad e ISO/IEC 25010 sin ocultar los elementos pendientes. También considero importante la reflexión ética aplicada al proyecto. Los datos de usuarios, credenciales y tokens no pueden tratarse como simples detalles técnicos: deben protegerse, excluirse del repositorio y evitarse en logs y capturas. De igual manera, reportar una disponibilidad o latencia no medida sería una forma de desinformación, incluso si el valor pareciera razonable. La automatización reduce algunos riesgos al ejecutar pruebas y conservar reportes de manera consistente. El documento final debe funcionar como evidencia profesional y no solo como requisito académico; por eso integra los resultados favorables y las limitaciones del experimento local, junto con planes concretos de mejora. Mi conclusión personal es que un producto profesional necesita ingeniería verificable y comunicación transparente. Ambas responsabilidades pertenecen al equipo completo y deben mantenerse durante la defensa, cuando cada integrante tendrá que explicar tanto los logros como los límites reales del sistema. En este cierre también reconozco que las mediciones del laboratorio no equivalen a producción: el uso de una base SQLite y un bloqueo compartido limita la comparación de estrategias. Mantener esa distinción evita atribuir garantías distribuidas a controles locales y permite orientar una siguiente iteración verificable.

14. Reflexión ética

El Código de Ética y Práctica Profesional de Ingeniería de Software de ACM/IEEE-CS [18] sitúa el interés público, la calidad del producto y el juicio profesional como responsabilidades

del desarrollo. En TiendaTech se traducen en decisiones concretas: no presentar pagos simulados como integración bancaria, no convertir un porcentaje de cobertura acotado en garantía de todo el sistema y no ocultar fallos para obtener una evaluación más favorable.

Las reservas y pagos tienen consecuencias potenciales para usuarios. Un reintento sin deduplicación puede descontar existencias de nuevo; una compensación incompleta puede mostrar una compra fallida con efectos persistentes. Aunque el proyecto sea académico, su documentación debe distinguir esos riesgos y explicar por qué no se recomienda usar un ensayo local como certificación productiva. Informar de un resultado negativo es una contribución técnica cuando impide una decisión basada en confianza falsa.

El manejo de credenciales y datos personales exige mínima exposición. Los secretos se suministran por configuración y no deben aparecer en commits, capturas ni logs. Antes de compartir evidencias se revisan tokens, direcciones y otros identificadores. Los datos sintéticos se rotulan como tales para evitar que se interpreten como transacciones de personas o estadísticas de un negocio real. La replicación no es una excusa para conservar innecesariamente información sensible.

La honestidad bibliográfica requiere comprobar que cada identificador corresponde a la obra citada. Reutilizar una referencia sin verificarla puede atribuir a un autor resultados que nunca publicó. Esta revisión corrigió la selección de fuentes académicas y mantuvo los estándares mediante enlaces institucionales, sin inventar DOI para cumplir una casilla formal.

El uso de IA generativa no transfiere la autoría ni la responsabilidad al asistente. Las sugerencias deben contrastarse con código y resultados; una redacción convincente puede ocultar supuestos incorrectos. Tampoco es admisible atribuir experiencias personales, firmas o aceptación a otros integrantes sin su revisión. Las reflexiones individuales se presentan para revisión y no sustituyen ese consentimiento.

Finalmente, la urgencia de entrega no justifica cambiar datos ni cerrar incidencias sin evidencia. Este informe conserva trabajo pendiente, acota estimaciones y no declara un porcentaje de similitud que no se ha medido. La obligación profesional es comunicar lo conocido y lo desconocido con suficiente claridad para que otra persona pueda evaluar el producto y continuar sin repetir errores.

15. Declaraciones

15.1. Autoría y contribución

El equipo AGLS está compuesto por los cuatro integrantes de la portada, con roles asignados desde la primera entrega. La tabla 13 resume responsabilidades y evidencia de trabajo colectivo, sin asignar porcentajes ni afirmar exclusividad sobre archivos desarrollados conjuntamente.

Tabla 13: Contribuciones por rol, sujetas a revisión final de sus autores.

Integrante	Contribución documentada
Jhinson Aucatoma	Arquitectura, ADR, modelo y distribución de datos; organización de evidencias de clúster y decisiones.
Jeremy Gaibor	Desarrollo e integración de servicios, comunicaciones y banco experimental; scripts y resultados incorporados al repositorio.
Andy Sanchez	Pruebas, cobertura, complejidad, CI, seguridad, carga e instrumentación; informes con alcance y límites.
José Lozano	Denominación y trazabilidad, revisión documental y de incidencias, consolidación bibliográfica y cierre acumulativo.

La contribución de una persona no excluye ayuda de otra. Los commits y las respuestas de revisión constituyen evidencia de evolución, no una medición automática de esfuerzo individual. Esta memoria no añade firmas digitales ni certifica una aprobación que no haya sido comunicada.

15.2. Uso de inteligencia artificial generativa

Las herramientas declaradas por el documentalista para el equipo son Claude para Jhinson y Andy, y Codex para Jeremy y José. No se incorporan otras herramientas generativas por suposición. Se usaron como apoyo para análisis técnico, desarrollo, revisión y redacción, no como sustitución del trabajo ni de la responsabilidad humana. Jhinson empleó Claude en materiales de arquitectura y decisiones, cuya coherencia con los ADR y el despliegue revisó personalmente; Jeremy empleó Codex en implementación y banco de pruebas, y revisó su ejecución, código y resultados; Andy empleó Claude en calidad, CI, seguridad y carga, y contrastó las afirmaciones con pruebas e informes; José empleó Codex en trazabilidad, revisión documental, bibliografía, tablas, figuras y LaTeX, y comprobó fuentes, rutas, compilación y presentación del PDF.

En el cierre documental, Codex asistió específicamente a José en la inspección de evidencias, conciliación de resultados, preparación de tablas y figuras, revisión bibliográfica, edición de LaTeX y comprobación del PDF. Las secciones afectadas abarcan arquitectura, diseño del estudio, resultados, calidad, gestión de la revisión, conclusiones y declaraciones. No se atribuye un modelo, versión, prompt o procedimiento de validación más específico que el comunicado por cada integrante.

Los textos individuales requieren lectura de cada autor antes de su presentación como declaración personal. La generación asistida no demuestra ausencia de errores: las afirmaciones se mantienen vinculadas a archivos, y los resultados faltantes se declaran como no medidos. La comprobación institucional de similitud inferior al 15 % no se ejecutó en este cierre y no se certifica.

15.3. Reproducibilidad documental

La fuente principal es `docs/entrega4/PFC4.tex` y utiliza la bibliografía compartida `docs/entrega3/referenciasPFC.bib`. La compilación se realiza desde `docs/entrega4/` mediante pdfLaTeX, Biber y dos pasadas adicionales de pdfLaTeX, según las instrucciones del README principal. Las imágenes se resuelven con rutas relativas y los diagramas C4 se generan desde TikZ.

Para reproducir el PDF desde un clon se versionan el logo `UteqLogo.png`, las imágenes de `img/` y las figuras `cierre/boxplot-p95.png` y `cierre/spark-t1.png`; estas últimas se regeneran con el script y los CSV del mismo directorio. El paquete experimental reúne en `experiments/paso7/coordination_lab.py` el generador, la carga concurrente, el inyector y el oráculo; conserva bancos y salidas en `experiments/paso7/evidence/`, las 120 corridas crudas y sus bases auditables en `experiments/paso8/resultados/`, y el cuaderno `experiments/paso8/analisis.ipynb`. `CITATION.cff` registra autoría y citación.

El README de esta entrega fija CPython 3.11.1, Matplotlib 3.9.0 y el identificador de la imagen TeX Live 2026; el cuaderno puede verificarse con el ejecutor estándar incluido, sin instalar Jupyter. También proporciona desde la clonación los comandos de prueba, regeneración, análisis y compilación. El 1 de septiembre de 2026, una clonación local aislada del árbol candidato completó tres pruebas, 120 corridas, el cuaderno, las figuras y la compilación de 39 páginas en 623,55 segundos (10 min 23,55 s), con 120/120 aprobaciones del oráculo y cero inconsistencias observadas. La descarga dependiente de la red se excluyó del cronómetro y el resultado no se equipara a una ejecución del sistema distribuido productivo.

El hash enlazado identifica las fuentes remotas de las mediciones. El cierre se integra en los nombres originales del documento y la bibliografía; las versiones anteriores permanecen consultables en el historial de Git.

16. Bibliografía

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*. Springer International Publishing, 2020. DOI: [10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2)
- [2] R. Taft et al., «CockroachDB: The Resilient Geo-Distributed SQL Database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, págs. 1493-1509. DOI: [10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134)
- [3] S. Gilbert y N. Lynch, «Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601)
- [4] E. Brewer, «CAP twelve years later: How the rules have changed,» *Computer*, vol. 45, n.º 2, págs. 23-29, 2012. DOI: [10.1109/MC.2012.37](https://doi.org/10.1109/MC.2012.37)
- [5] D. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33)
- [6] L. Lamport, «Time, clocks, and the ordering of events in a distributed system,» *Communications of the ACM*, vol. 21, n.º 7, págs. 558-565, 1978. DOI: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563)
- [7] G. M. Amdahl, «Validity of the single processor approach to achieving large scale computing capabilities,» en *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*, ACM Press, 1967, pág. 483. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560)
- [8] J. L. Gustafson, «Reevaluating Amdahl's law,» *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: [10.1145/42411.42415](https://doi.org/10.1145/42411.42415)
- [9] M. Rigger y Z. Su, «Finding bugs in database systems via query partitioning,» *Proceedings of the ACM on Programming Languages*, vol. 4, n.º OOPSLA, págs. 1-30, 2020. DOI: [10.1145/3428279](https://doi.org/10.1145/3428279)
- [10] G. Marquez, F. Osses y H. Astudillo, «Review of Architectural Patterns and Tactics for Microservices in Academic and Industrial Literature,» *IEEE Latin America Transactions*, vol. 16, n.º 9, págs. 2321-2327, 2018. DOI: [10.1109/TLA.2018.8789551](https://doi.org/10.1109/TLA.2018.8789551)
- [11] C. Zhong et al., «Domain-Driven Design for Microservices: An Evidence-Based Investigation,» *IEEE Transactions on Software Engineering*, vol. 50, n.º 6, págs. 1425-1449, 2024. DOI: [10.1109/TSE.2024.3385835](https://doi.org/10.1109/TSE.2024.3385835)
- [12] P. Nawrocki, K. Wrona, M. Marczak y B. Sniezynski, «A Comparison of Native and Cross-Platform Frameworks for Mobile Applications,» *Computer*, vol. 54, n.º 3, págs. 18-27, 2021. DOI: [10.1109/MC.2020.2983893](https://doi.org/10.1109/MC.2020.2983893)
- [13] M. Shahin, M. Ali Babar y L. Zhu, «Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices,» *IEEE Access*, vol. 5, págs. 3909-3943, 2017. DOI: [10.1109/ACCESS.2017.2685629](https://doi.org/10.1109/ACCESS.2017.2685629)
- [14] P. Rostami Mazrae, T. Mens, M. Golzadeh y A. Decan, «On the usage, co-usage and migration of CI/CD tools: A qualitative analysis,» *Empirical Software Engineering*, vol. 28, n.º 2, 2023. DOI: [10.1007/s10664-022-10285-5](https://doi.org/10.1007/s10664-022-10285-5)

- [15] M. Hui et al., «Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,» *Journal of Systems and Software*, vol. 220, pág. 112 232, 2025. DOI: [10.1016/j.jss.2024.112232](https://doi.org/10.1016/j.jss.2024.112232)
- [16] ISO/IEC. «ISO/IEC 25010:2023: Product quality model,» visitado 1 de sep. de 2026. dirección: <https://www.iso.org/standard/78176.html>
- [17] ISO/IEC. «ISO/IEC 25023:2016: Measurement of system and software product quality,» visitado 1 de sep. de 2026. dirección: <https://www.iso.org/standard/35747.html>
- [18] ACM/IEEE-CS. «Software Engineering Code of Ethics and Professional Practice, Version 5.2,» visitado 1 de sep. de 2026. dirección: <https://www.acm.org/code-of-ethics/software-engineering-code>